Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 3

Técnicas de Búsqueda por Trayectorias
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Componentes Comunes a los Algoritmos de Trayectoria **3**
2.1. Representación de soluciones y operador de reparación 3
2.2. Operador de vecindad (usado en SA) 3
2.3. Operador de mutación del 20 % (usado en ILS e ILS-ES) 3
2.4. Búsqueda Local (BL) como subrutina 3

3. Algoritmos de Búsqueda por Trayectorias **4**
3.1. Enfriamiento Simulado (SA) 4
3.2. Búsqueda Multiarranque Básica (BMB) 5
3.3. Búsqueda Local Reiterada (ILS) 6
3.4. ILS con Enfriamiento Simulado (ILS-ES) 6

4. Estructura del Código y Manual de Usuario **7**

5. Experimentos y Resultados **8**
5.1. Casos del problema 8
5.2. Resultados 8
5.3. Boxplots 9

6. Análisis de Resultados **10**
6.1. Comportamiento de SA: escala con N y eficiencia 10
6.2. Comportamiento de BMB: impacto del tamaño del vecindario 10
6.3. Comportamiento de ILS: perturbación vs reinicio 11
6.4. Comportamiento de ILS-ES: hibridación SA + perturbación 11
6.5. Comparativa general P1-P2 vs P3 11
6.6. Análisis de la variabilidad (σ) 12
6.7. Análisis de los boxplots 12
6.8. Capacidad de generalización (test 2025) 12
6.9. Conclusiones generales 13

7. Trabajo Voluntario: Estudio de los Parámetros de SA **13**
7.1. Invarianza de fase: por qué φ no afecta a los resultados 13
7.2. Resultados: estudio de φ 14
7.3. Resultados: estudio de k_{vec} 15
7.4. Análisis del estudio de k_{vec} 16
7.5. Conclusiones del estudio de parámetros 16

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y $\lambda = 500$.

Algorithm 1: Cálculo de fitness(w)

Entrada: pesos w, beneficio por activo *benefit_factor*[], matriz de covarianzas *cov*[][], λ
Salida: valor de fitness

```
benefit ← 0;
para i ← 1 to N hacer
    benefit ← benefit + w[i] · benefit_factor[i];
risk_sq ← 0;
para i ← 1 to N hacer
    para j ← 1 to N hacer
        risk_sq ← risk_sq + w[i] · cov[i][j] · w[j];
devolver benefit − λ · √risk_sq
```

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i$$

2. Componentes Comunes a los Algoritmos de Trayectoria

2.1. Representación de soluciones y operador de reparación

Una solución es un vector $w \in \mathbb{R}^N$ donde $w_i \in \{0\} \cup [lo_i, hi_i]$ indica la fracción de capital invertida en el activo i. La función `fix()` garantiza la factibilidad tras cualquier modificación: normaliza los pesos para que sumen 1, recorta a $[lo, hi]$ y redistribuye iterativamente.

2.2. Operador de vecindad (usado en SA)

El operador de vecino de SA modifica **una única posición** del vector de pesos: se elige aleatoriamente una posición *pos* y se le asigna un valor nuevo $w_{pos} \sim U(lo, hi)$. A continuación se aplica `fix()` para restaurar la factibilidad.

Algorithm 2: generarVecino(*sol*, problema)

Entrada: solución *sol*, $[lo, hi]$
Salida: solución vecina

```
pos ← aleatorio en [0, N − 1];
sol' ← sol;
sol'[pos] ← aleatorio en [lo, hi];
fix(sol');
devolver sol'
```

2.3. Operador de mutación del 20 % (usado en ILS e ILS-ES)

El operador de mutación selecciona $n_{mut} = \text{máx}(1, \lfloor 0{,}2 \cdot N \rfloor)$ posiciones al azar, extrae sus valores, los baraja y los reasigna a las mismas posiciones. Finalmente aplica `fix()`. Este operador produce una perturbación más drástica que un único cambio de gen, escapando de cuencas de atracción locales.

Algorithm 3: mutar20 %(*sol*, problema)

Entrada: solución *sol*, $[lo, hi]$
Salida: solución mutada

```
n_mut ← máx(1, ⌊0,2 · N⌋);
Barajar índices [0, N − 1] aleatoriamente → indices;
valores ← [sol[indices[0]], ..., sol[indices[n_mut − 1]]];
Barajar valores aleatoriamente;
para i ← 0 to n_mut − 1 hacer
    sol'[indices[i]] ← valores[i];
fix(sol');
devolver sol'
```

2.4. Búsqueda Local (BL) como subrutina

Los algoritmos BMB, ILS e ILS-ES usan la BL de la Práctica 1 como subrutina interna: operador de intercambio con proporción $p = 0{,}4$, primer-mejor, y llamada a `fix()` en

cada candidato. La BL termina cuando no hay mejora en toda una pasada aleatoria del vecindario, o cuando se agota el presupuesto de evaluaciones.

Algorithm 4: BL($sol_inicial$, problema, maxEvals)

Entrada: solución inicial, problema, maxEvals
Salida: solución, fitness, evaluaciones

```
1  Pares ← {(i, j) : i ≠ j};   sol ← sol_inicial;
2  fit ← evaluar(sol);   mejora ← verdadero;   evals ← 1;
3  mientras mejora y evals < maxEvals hacer
4      mejora ← falso;   Barajar Pares;
5      para (ori, dst) en Pares y evals < maxEvals hacer
6          Generar vecino por transferencia de p · sol[ori] de ori a dst;
7          evals ← evals + 1;
8          si fit' > fit entonces
9              sol ← sol';;
10             fit ← fit';;
11             mejora ← verdadero;;
12             salir;
13 devolver sol, fit, evals
```

3. Algoritmos de Búsqueda por Trayectorias

3.1. Enfriamiento Simulado (SA)

Parámetros

Parámetro	Valor	Significado
φ	0,3	Prob. de aceptar un vecino μ peor que la solución inicial
μ	0,2	Fracción de degradación aceptable para calcular T_0
T_f	10^{-3}	Temperatura final
k_{vec}	10	Factor: max_vecinos $= k_{vec} \cdot N$
k_{exit}	1	Factor: max_éxitos $= k_{exit} \cdot N$

Temperatura inicial y parámetros de enfriamiento

La temperatura inicial T_0 se calibra para que, con la solución aleatoria inicial s_0, la probabilidad de aceptar un vecino μ-peor sea φ:

$$T_0 = \frac{\mu \cdot |\text{fitness}(s_0)|}{-\ln(\varphi)}$$

El número de iteraciones de enfriamiento es $M = \text{máx}\left(1, \left\lfloor \frac{\text{maxEvals}}{\text{max_vecinos}} \right\rfloor\right)$, y el coeficiente de Cauchy modificado:

$$\beta = \frac{T_0 - T_f}{M \cdot T_0 \cdot T_f}$$

de modo que $T_{k+1} = \dfrac{T_k}{1 + \beta \cdot T_k}$.

Pseudocódigo

Algorithm 5: SA (Enfriamiento Simulado)

Entrada: problema, maxEvals
Salida: mejor solución

$sol \leftarrow$ `crearSolución()`; $fit \leftarrow$ evaluar(sol); $mejor \leftarrow sol$;
$T_0 \leftarrow (\mu \cdot |fit|)/(-\ln \varphi)$; $M \leftarrow$ máx$(1, \lfloor \text{maxEvals}/(k_{vec} \cdot N) \rfloor)$;
$\beta \leftarrow (T_0 - T_f)/(M \cdot T_0 \cdot T_f)$; $T \leftarrow T_0$; $evals \leftarrow 1$;
para $enfr \leftarrow 1$ **to** M ***y*** $evals < maxEvals$ **hacer**
  $n_vec \leftarrow 0$; $n_exit \leftarrow 0$;
  mientras $n_vec < k_{vec} \cdot N$ ***y*** $n_exit < k_{exit} \cdot N$ ***y*** $evals < maxEvals$ **hacer**
    $sol' \leftarrow$ `generarVecino(`sol, $problema$`)`; $fit' \leftarrow$ evaluar(sol'); $evals$ += 1;
    $\Delta \leftarrow fit' - fit$;
    si $\Delta > 0$ ***o*** $U(0,1) < e^{\Delta/T}$ **entonces**
      $sol \leftarrow sol'$; $fit \leftarrow fit'$; n_exit += 1;
      si $fit > fit(mejor)$ **entonces**
        $mejor \leftarrow sol$;
    n_vec += 1;
  si $n_exit = 0$ **entonces**
    terminar;
  $T \leftarrow T/(1 + \beta \cdot T)$;
  si $T < T_f$ **entonces**
    terminar;
devolver $mejor$

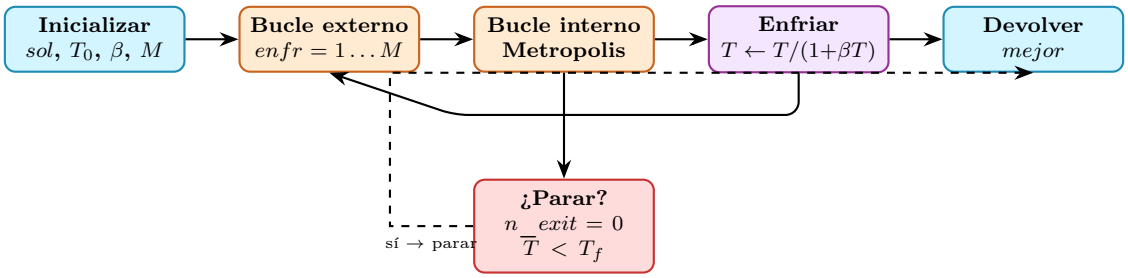


Figura 1: Estructura de SA. El bucle externo controla el enfriamiento; el interno genera y acepta vecinos mediante el criterio de Metropolis.

3.2. Búsqueda Multiarranque Básica (BMB)

BMB ejecuta $n_{iter} = 5$ arranques independientes: cada uno genera una solución aleatoria y la refina con BL hasta un máximo de 2 000 evaluaciones. Al final devuelve la mejor de las 5 soluciones encontradas.

Algorithm 6: BMB (Búsqueda Multiarranque Básica)

Entrada: problema, maxEvals
Salida: mejor solución

$mejor \leftarrow -\infty$; $evals \leftarrow 0$;
para $i \leftarrow 1$ **to** $n_{iter} = 5$ **hacer**
  si $evals \geq maxEvals$ **entonces**
    salir;
  $budget \leftarrow$ mín(2000, maxEvals $- evals$);
  $sol \leftarrow$ `crearSolución()`;
  $res \leftarrow$ `BL(`*sol, problema, budget*`)`;
  $evals \leftarrow evals + res.evals$;
  si $res.fitness > mejor.fitness$ **entonces**
    $mejor \leftarrow res$;
devolver $mejor$

3.3. Búsqueda Local Reiterada (ILS)

ILS aplica BL 5 veces: la primera sobre una solución aleatoria y las 4 restantes sobre mutaciones (20 % de posiciones barajadas) de la **mejor solución encontrada** hasta ese momento (criterio *best*). Cada BL tiene un presupuesto de 2 000 evaluaciones.

Algorithm 7: ILS (Búsqueda Local Reiterada)

Entrada: problema, maxEvals
Salida: mejor solución

$mejor \leftarrow$ `BL(crearSolución(),` *problema,* 2000`)`;
$evals \leftarrow mejor.evals$;
para $i \leftarrow 1$ **to** $n_{iter} - 1 = 4$ **hacer**
  $sol' \leftarrow$ `mutar20%(`*mejor.sol, problema*`)`;
  $res \leftarrow$ `BL(`*sol′, problema,* 2000`)`;
  $evals \leftarrow evals + res.evals$;
  si $res.fitness > mejor.fitness$ **entonces**
    $mejor \leftarrow res$;
devolver $mejor$

3.4. ILS con Enfriamiento Simulado (ILS-ES)

ILS-ES sustituye la BL de ILS por SA como mecanismo de búsqueda local. El presupuesto total se divide en partes iguales: cada SA recibe $\lfloor \text{maxEvals}/5 \rfloor$ evaluaciones. El mismo operador de mutación del 20 % genera la perturbación entre iteraciones.

Algorithm 8: ILS-ES (ILS con Enfriamiento Simulado como subalgoritmo)

Entrada: problema, maxEvals
Salida: mejor solución

$budget \leftarrow \lfloor \text{maxEvals}/n_{iter} \rfloor$; $evals \leftarrow 0$;
$mejor \leftarrow$ `SA(crearSolución(), problema, budget)`; $evals \leftarrow mejor.evals$;
para $i \leftarrow 1$ **to** $n_{iter} - 1 = 4$ **hacer**
    $sol' \leftarrow$ `mutar20%(`$mejor.sol$, $problema$`)`;
    $res \leftarrow$ `SA(`sol', $problema$, $budget$`)`;
    $evals \leftarrow evals + res.evals$;
    si $res.fitness > mejor.fitness$ **entonces**
        $mejor \leftarrow res$;
devolver $mejor$

4. Estructura del Código y Manual de Usuario

```
portfolio/
  main.cpp              -- Comparativa P1+P2 (14 algoritmos)
  main_p3.cpp           -- Comparativa P3 (8 algoritmos)
  main_sa_params.cpp    -- Estudio parametros SA (vol.)
  main_blx_alpha.cpp    -- Estudio alpha BLX (Practica 2)
  CMakeLists.txt      -- Targets: main, main_p3, study_sa_params
  boxplot.py          -- Genera boxplots PNG desde los CSV
  data/
    ibex_35.csv, syp_100.csv, syp_500.csv
  src/
    portfolioProblem.cpp -- fitness, fix, createSolution
    sa.cpp  bmb.cpp  ils.cpp  ils_es.cpp
    localsearch.cpp  genetic.cpp  greedy.cpp
  inc/
    sa.h  bmb.h  ils.h  ils_es.h  localsearch.h
  common/
    mh.h  problem.h  solution.h  random.hpp
```

Compilación y ejecución:

```
cmake -S . -B build && cmake --build build -j$(nproc)

# Comparativa P3 (50 semillas, 10000 evals)
./build/main_p3 10000 50

# Estudio parametros SA (trabajo voluntario)
./build/study_sa_params 10000 50

# Generar boxplots (requiere matplotlib, seaborn, pandas)
python3 boxplot.py
```

Los CSVs generados son: `p3_results_*.csv` (boxplots), `p3_stats_*.csv` (estadísticas para tablas), `sa_phi_*.csv` y `sa_kvecinos_*.csv` (trabajo voluntario).

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	N	Días train	Días test	$[lo, hi]$
`ibex_35.csv`	30	2559	254	[0,005, 0,08]
`syp_100.csv`	97	2516	249	[0,005, 0,05]
`syp_500.csv`	457	2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, MAX_EVALS= 10 000, semillas 1–50. Los algoritmos de referencia son Greedy y Random (P1 sin optimización), BL (P1, operador de intercambio) y AM-Best (P2, mejor algoritmo).

5.2. Resultados

Cuadro 1: IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
— Referencias (P1 y P2) —							
Greedy	−4,650	0,000	−3,387	0,000	0,115	1,0	0,000
Random	−4,941	0,070	−3,306	0,104	0,036	10000,0	0,026
BL	−4,573	0,019	−3,109	0,008	0,022	1257,0	0,001
— Mejor P2 —							
AM-Best	**−4,491**	0,001	**−3,084**	0,003	0,033	10000,4	0,011
— P3 nuevos —							
BMB	−4,546	0,014	−3,102	0,008	0,023	6041,3	0,004
ILS	−4,535	0,037	−3,096	0,014	0,026	5070,2	0,003
SA	−4,605	0,036	−3,144	0,042	0,034	2841,9	0,001
ILS-ES	−4,595	0,031	−3,143	0,040	0,034	8160,0	0,003

Cuadro 2: S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
— Referencias (P1 y P2) —							
Greedy	−3,664	0,000	−5,020	0,000	0,010	1,0	0,000
Random	−3,390	0,055	−4,113	0,193	0,080	10000,0	0,059
BL	−3,032	0,067	−3,870	0,033	0,097	5258,5	0,027
— Mejor P2 —							
AM-Best	−2,912	0,025	−3,725	0,033	0,094	10000,5	0,120
— P3 nuevos —							
BMB	−2,957	0,041	−3,845	0,028	0,094	10000,0	0,091
ILS	−2,983	0,098	−3,851	0,046	0,096	10000,0	0,053
SA	**−2,870**	0,015	**−3,780**	0,026	0,096	9091,0	0,015
ILS-ES	−2,885	0,011	−3,786	0,022	**0,098**	9172,6	0,015

Cuadro 3: S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
— Referencias (P1 y P2) —							
Greedy	−3,438	0,000	−5,205	0,000	−0,063	1,0	0,000
Random	−3,642	0,042	−4,430	0,167	0,012	10000,0	1,300
BL	−3,318	0,164	−4,228	0,119	0,008	9839,7	0,679
— Mejor P2 —							
AM-Best	−2,811	0,031	−3,907	0,065	0,017	10000,4	1,702
— P3 nuevos —							
BMB	−3,195	0,072	−4,127	0,062	0,011	10000,0	1,747
ILS	−2,819	0,095	−4,004	0,054	0,015	10000,0	1,192
SA	**−2,627**	0,009	−3,917	0,029	0,022	9140,9	0,313
ILS-ES	−2,709	0,016	**−3,895**	0,035	**0,022**	10000,0	0,326

5.3. Boxplots

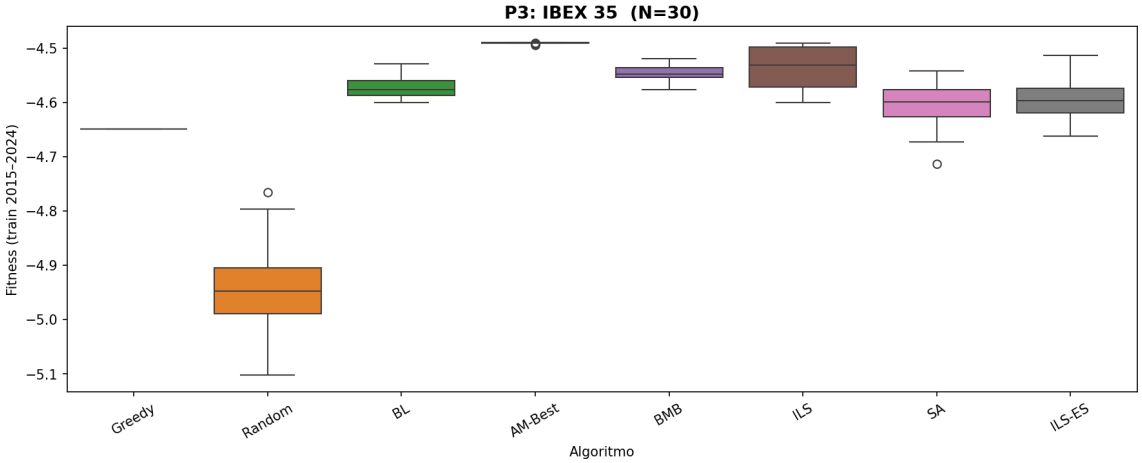


Figura 2: Distribución del fitness de entrenamiento (2015–2024) en IBEX 35 para los 8 algoritmos.

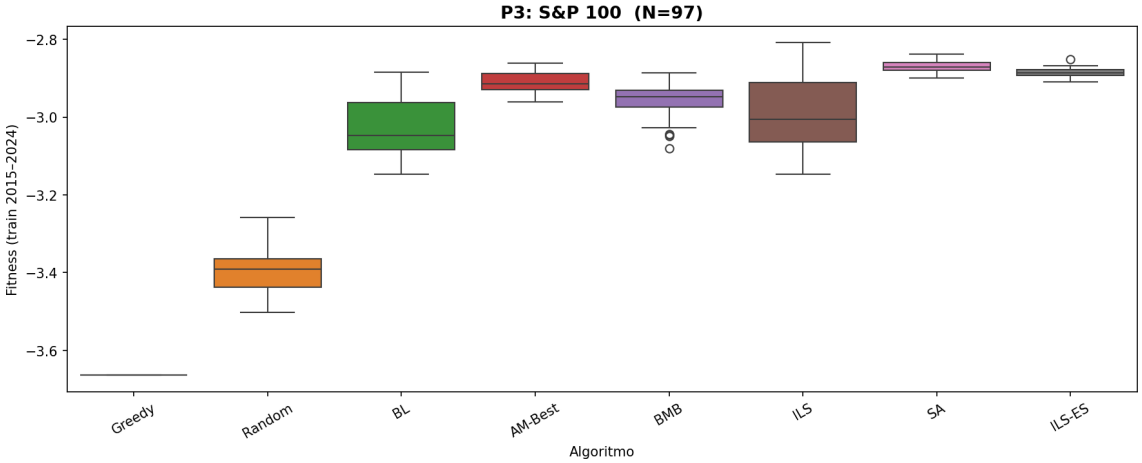


Figura 3: Distribución del fitness de entrenamiento en S&P 100.

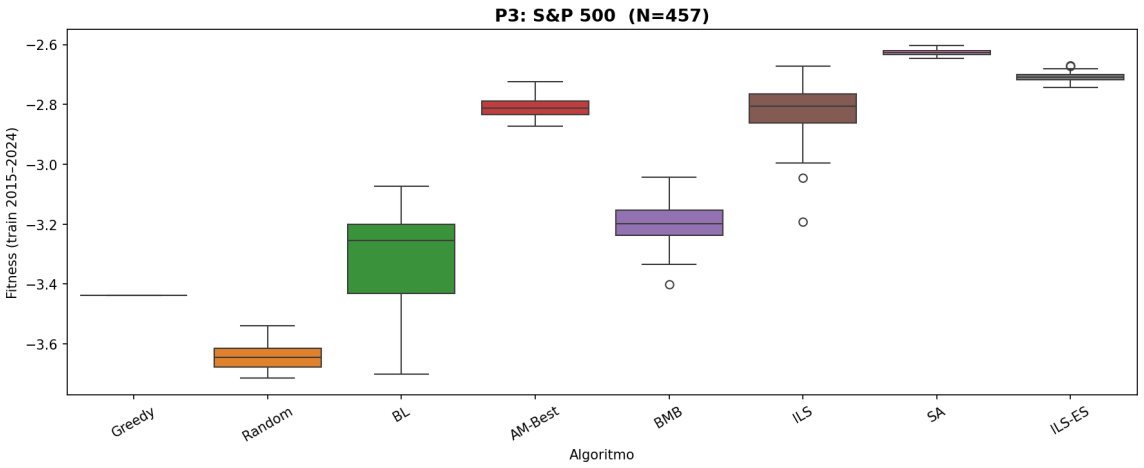


Figura 4: Distribución del fitness de entrenamiento en S&P 500.

6. Análisis de Resultados

6.1. Comportamiento de SA: escala con N y eficiencia

SA es el **algoritmo más eficaz en syp_100 y syp_500**. En syp_500 obtiene un fitness medio de $-2{,}627$ con $\sigma = 0{,}009$ — la **menor varianza de todos los algoritmos en ese dataset** —, superando incluso al mejor algoritmo de la Práctica 2 (AM-Best: $-2{,}811$) con solo un 3 % del tiempo de ejecución (0,313 s vs 1,702 s).

La causa es la naturaleza del operador de vecindad: un único cambio de gen produce candidatos de baja coste computacional ($O(N)$ en la evaluación incremental), permitiendo explorar miles de vecinos dentro del presupuesto. En cambio, BL necesita revisar $N(N-1) = 208\,392$ pares por pasada, agotando el presupuesto sin llegar a convergencia.

SA usa en media 9 141 evaluaciones, no las 10 000 disponibles: el criterio de parada anticipada (*num_éxitos*$= 0$ en un enfriamiento) indica que la temperatura ha descendido al punto en que el algoritmo ya no encuentra vecinos mejorantes ni los acepta por probabilidad, señal de convergencia real.

En **ibex_35** ($N = 30$), SA obtiene solo $-4{,}605$, siendo el peor de los P3 por detrás de BMB e ILS. El problema es la calibración del β: con $N = 30$, max_vec$= 300$, $M = 33$ enfriamientos, T_0 muy cercano a cero (fitness $\approx -4{,}6$), la temperatura desciende rápidamente y el algoritmo converge antes de explorar suficiente.

6.2. Comportamiento de BMB: impacto del tamaño del vecindario

BMB funciona bien en problemas pequeños: en ibex_35 obtiene $-4{,}546$, comparable a ILS ($-4{,}535$). Sin embargo, **fracasa dramáticamente en syp_500**: $-3{,}195$, peor incluso que la BL simple ($-3{,}318$ ya es malo, pero BMB es aún peor).

La razón es directa: BMB aplica BL con presupuesto fijo de 2 000 evaluaciones por arranque. En syp_500 ($N = 457$), una pasada completa del vecindario requiere $457 \times 456 = 208\,392$ evaluaciones, por lo que con 2 000 evals la BL solo explora el 1 % del vecindario y para siempre por límite de presupuesto, sin llegar jamás a convergencia. Los 5 arranques

son 5 BLs sin converger, y la "mejor de 5" no mejora sustancialmente sobre una sola BL sin converger.

La varianza de BMB en syp_500 ($\sigma = 0{,}072$) es moderada: los 5 arranques aleatorios cubren distintas regiones pero ninguna llega a explotarse correctamente.

6.3. Comportamiento de ILS: perturbación vs reinicio

ILS supera a BMB en todos los datasets, pero la diferencia es especialmente marcada en syp_500: ILS obtiene $-2{,}819$ frente a $-3{,}195$ de BMB.

La clave es el **operador de mutación del 20 %**: en lugar de reiniciar desde un punto aleatorio (BMB), ILS parte de la mejor solución actual y la perturba manteniendo $N - 0{,}2N = 0{,}8N$ posiciones intactas. Esto significa que la BL posterior comienza en una región del espacio próxima a un buen óptimo local, pudiendo encontrar uno aún mejor en las 2 000 evaluaciones disponibles. En BMB, una solución aleatoria en syp_500 está tan lejos de cualquier óptimo que 2 000 evals son insuficientes para llegar a ninguno.

ILS tiene alta varianza en syp_500 ($\sigma = 0{,}095$): la perturbación del 20 % puede llevar a zonas muy diferentes del espacio y el resultado depende de si la BL consigue explotar esa zona dentro del presupuesto.

6.4. Comportamiento de ILS-ES: hibridación SA + perturbación

ILS-ES obtiene resultados intermedios entre SA e ILS en syp_500 (fitness $-2{,}709$, mejor que ILS $-2{,}819$ pero peor que SA puro $-2{,}627$). Sin embargo, ILS-ES destaca en **capacidad de generalización**: en Ben.2025, ILS-ES iguala a SA (0,022) en syp_500 con menor varianza en test ($\sigma = 0{,}035$ vs 0,029), y supera a SA en syp_100 (Ben.2025: 0,098 vs 0,096).

La diversificación que aporta la mutación del 20 % entre llamadas a SA parece ayudar a encontrar soluciones que generalizan mejor, aunque con un ligero coste en fitness de entrenamiento. Cada iteración de SA dispone de $\lfloor 10\,000/5 \rfloor = 2\,000$ evaluaciones, que para syp_500 equivalen a los mismos $M = \max(1, \lfloor 2000/4570 \rfloor) = 1$ enfriamiento con 4 570 vecinos máximos, siendo en práctica una ejecución SA muy corta.

La varianza de ILS-ES en syp_500 ($\sigma = 0{,}016$) es la segunda más baja tras SA ($\sigma = 0{,}009$), lo que indica que la perturbación no introduce inestabilidad excesiva respecto a SA puro.

6.5. Comparativa general P1-P2 vs P3

Dataset	Mejor P1	Mejor P2	Mejor P3	Mejora P3 vs P2
ibex_35	BL ($-4{,}573$)	AM-Best ($-4{,}491$)	ILS ($-4{,}535$)	$-0{,}044$
syp_100	BL ($-3{,}032$)	AM-Best ($-2{,}912$)	SA ($-2{,}870$)	$+0{,}042$
syp_500	BL ($-3{,}318$)	AM-Best ($-2{,}811$)	SA ($-2{,}627$)	$+0{,}184$

El patrón más importante es la **inversión de ventaja con** N. Para N pequeño (ibex_35), AM-Best (P2) sigue siendo el mejor algoritmo: la exploración global de la población evolutiva y la BL sobre los mejores individuos es insuperable en espacios pequeños. Ningún algoritmo de P3 supera a AM-Best en ibex_35.

Para N grande (syp_100 y especialmente syp_500), los algoritmos de trayectoria de P3 superan a los evolutivos de P2. SA supera a AM-Best en 0,184 unidades de fitness en syp_500 — una diferencia considerable. La razón es que, en alta dimensión, la evaluación de fitness es $O(N^2)$ (por la covarianza), lo que hace que 10 000 evaluaciones se consuman rápidamente. SA, con su operador de vecindad simple ($O(1)$ para generar el vecino), aprovecha mejor el presupuesto explorando un número mayor de puntos del espacio.

6.6. Análisis de la variabilidad (σ)

La variabilidad refleja la sensibilidad de cada algoritmo a la semilla inicial:

- **SA**: σ mínima (0,009 en syp_500). El enfriamiento progresivo produce convergencia robusta independiente de la solución inicial.
- **ILS-ES**: segunda menor varianza (0,011–0,016). La combinación SA + perturbación estabiliza la búsqueda.
- **BMB**: varianza moderada. Los 5 reinicios aleatorios cubren distintas zonas pero no convergen en problemas grandes.
- **ILS**: mayor varianza de P3 en syp_500 ($\sigma = 0{,}095$), similar a BL ($\sigma = 0{,}164$ es el peor). La perturbación del 20 % puede llevar a zonas muy distintas con resultados inconsistentes.
- **AM-Best (P2)**: $\sigma = 0{,}001$ en ibex_35 — prácticamente determinístico —, pero $\sigma = 0{,}031$ en syp_500.

6.7. Análisis de los boxplots

Los boxplots (Figuras 2–4) confirman el análisis:

- En **ibex_35**, AM-Best domina con caja casi imperceptible (determinístico). ILS y BMB tienen medianas próximas pero con cajas más anchas. SA e ILS-ES se agrupan claramente por debajo de AM-Best, con cajas similares entre sí (ambos usan SA como núcleo). Greedy, aunque determinístico, aparece en la parte baja.
- En **syp_100**, SA e ILS-ES aparecen visiblemente por encima de AM-Best, con cajas muy compactas. BMB e ILS tienen medianas parecidas entre sí pero peores que SA/ILS-ES. BL está claramente por debajo de todos los algoritmos P3. La escala visual permite apreciar la diferencia entre trayectorias (SA/ILS-ES) y el resto.
- En **syp_500**, la separación entre grupos es máxima: SA tiene la mediana más alta y la caja más pequeña de todos los algoritmos. ILS-ES está inmediatamente por debajo. Después viene el grupo AM-Best/ILS. BMB y BL aparecen claramente separados hacia abajo. Random y Greedy son los peores.

6.8. Capacidad de generalización (test 2025)

La columna Ben.2025 (beneficio real en el año de test) muestra un patrón consistente: los algoritmos con mejor fitness de entrenamiento no siempre generalizan mejor.

Algoritmo	Train (syp_500)	Test 2025 (syp_500)	Ben.2025
Greedy	−3,438	−5,205	−0,063
BL	−3,318	−4,228	0,008
AM-Best	−2,811	−3,907	0,017
ILS	−2,819	−4,004	0,015
SA	−2,627	−3,917	0,022
ILS-ES	−2,709	−3,895	**0,022**

SA obtiene el mejor fitness de entrenamiento (−2,627) pero ILS-ES obtiene el mejor Ben.2025 (0,022). Esto sugiere que la diversificación de ILS-ES evita un leve sobreajuste al periodo de entrenamiento que SA puede presentar.

6.9. Conclusiones generales

1. **SA es el mejor algoritmo para N grande**: supera a todos (incluyendo AM-Best de P2) en syp_100 y syp_500, con la menor varianza. Su ventaja crece con N debido a la eficiencia de su operador de vecindad ($O(1)$ generación vs $O(N^2)$ por pasada en BL).
2. **BMB fracasa en alta dimensión**: el presupuesto de 2 000 evals/BL es insuficiente para convergencia en syp_500 (208 392 vecinos). Un único reinicio en una zona aleatoria nunca llega a explotarse correctamente.
3. **ILS supera a BMB** porque la perturbación del 20 % sobre la mejor solución mantiene la búsqueda en zonas prometedoras, mientras que el reinicio aleatorio de BMB pierde ese contexto.
4. **ILS-ES ofrece el mejor equilibrio generalización/consistencia**: menor varianza que ILS, mejor Ben.2025 que SA puro, con tiempo de cómputo reducido.
5. **AM-Best (P2) sigue siendo el mejor para N pequeño**: en ibex_35 ningún algoritmo de P3 lo supera. La exploración poblacional es más valiosa en espacios pequeños con presupuesto suficiente para convergencia de la BL.

7. Trabajo Voluntario: Estudio de los Parámetros de SA

Se ha implementado el ejecutable `study_sa_params` que realiza un barrido sistemático de los parámetros φ (probabilidad de aceptación inicial) y k_{vec} (factor de vecindad), con 50 semillas por configuración sobre los tres datasets.

7.1. Invarianza de fase: por qué φ no afecta a los resultados

Teóricamente, φ controla la temperatura inicial: $T_0 = \mu \cdot |\text{fitness}|/(-\ln \varphi)$. Valores mayores de φ producen mayores T_0 (mayor exploración inicial).

Sin embargo, el **esquema de Cauchy modificado presenta invarianza de fase**: pasado el primer enfriamiento, la temperatura converge a $T_k \approx 1/(\beta \cdot k)$ independientemente de T_0. Para el segundo paso: $T_2 = T_1/(1+\beta T_1) = T_0/[(1+\beta T_0)(1+\beta \cdot T_0/(1+\beta T_0))]$,

que para T_0 grande se aproxima a $1/(\beta \cdot 2)$ — independiente de T_0, y por tanto de φ.

La consecuencia práctica es que φ solo afecta al primer enfriamiento ($k = 1$), y las $M - 1$ iteraciones restantes convergen al mismo comportamiento sin importar el valor de φ.

7.2. Resultados: estudio de φ

Cuadro 4: Estudio de φ en SA — IBEX 35 ($N = 30$, mu=0.2, $T_f = 10^{-3}$, $k_{vec} = 10$)

φ	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,05	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
0,10	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
0,20	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
0,30	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
0,50	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
0,70	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
0,90	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341

Cuadro 5: Estudio de φ en SA — S&P 100 ($N = 97$)

φ	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,05	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
0,10	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
0,20	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
0,30	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
0,50	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
0,70	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
0,90	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963

Cuadro 6: Estudio de φ en SA — S&P 500 ($N = 457$)

φ	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,05	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
0,10	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
0,20	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
0,30	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
0,50	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
0,70	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
0,90	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217

Los tres datasets exhiben la misma **invarianza perfecta**: los 7 valores de φ producen exactamente los mismos resultados (hasta el último decimal), confirmando empíricamente la propiedad teórica de la invarianza de fase de Cauchy.

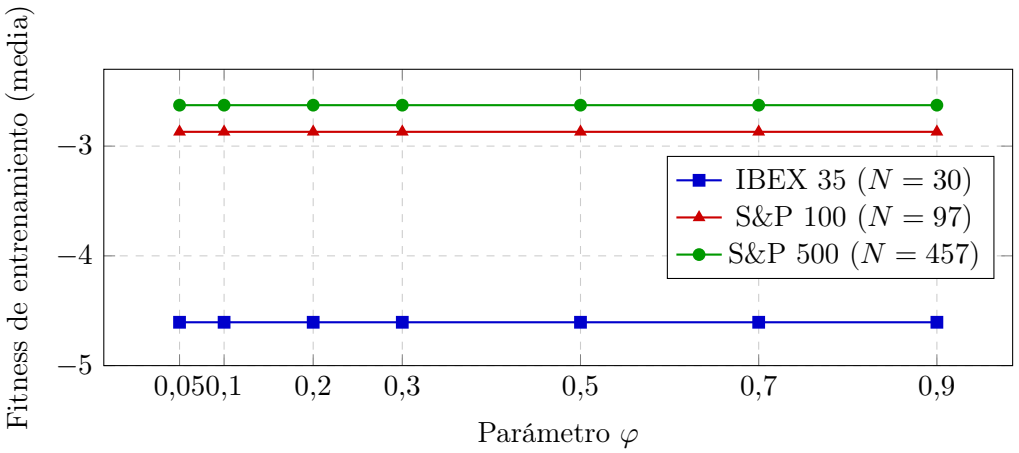


Figura 5: Fitness de entrenamiento de SA en función de φ para los tres datasets. Las tres líneas son perfectamente horizontales: φ no tiene ningún efecto práctico debido a la invarianza de fase del esquema de enfriamiento de Cauchy modificado.

7.3. Resultados: estudio de k_{vec}

Cuadro 7: Estudio de k_{vec} en SA — IBEX 35 ($N = 30$, $\varphi = 0{,}3$, mu=0.2, $T_f = 10^{-3}$)

k_{vec}	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
2	$-4{,}6901 \pm 0{,}0627$	$-3{,}1770 \pm 0{,}0495$	0,0349
5	$-4{,}6264 \pm 0{,}0401$	$-3{,}1513 \pm 0{,}0416$	0,0342
10	$-4{,}6051 \pm 0{,}0358$	$-3{,}1439 \pm 0{,}0418$	0,0341
15	$-4{,}5935 \pm 0{,}0299$	$-3{,}1394 \pm 0{,}0412$	0,0337
20	$-4{,}5916 \pm 0{,}0301$	$-3{,}1385 \pm 0{,}0413$	0,0336
30	$-4{,}5856 \pm 0{,}0262$	$-3{,}1359 \pm 0{,}0406$	0,0339

Cuadro 8: Estudio de k_{vec} en SA — S&P 100 ($N = 97$)

k_{vec}	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
2	$-2{,}8866 \pm 0{,}0152$	$-3{,}7866 \pm 0{,}0227$	0,0959
5	$-2{,}8725 \pm 0{,}0148$	$-3{,}7803 \pm 0{,}0231$	0,0964
10	$-2{,}8699 \pm 0{,}0148$	$-3{,}7801 \pm 0{,}0256$	0,0963
15	$-2{,}8737 \pm 0{,}0149$	$-3{,}7808 \pm 0{,}0248$	0,0962
20	$-2{,}8731 \pm 0{,}0149$	$-3{,}7811 \pm 0{,}0245$	0,0962
30	$-2{,}8830 \pm 0{,}0177$	$-3{,}7851 \pm 0{,}0240$	0,0958

Cuadro 9: Estudio de k_{vec} en SA — S&P 500 ($N = 457$)

k_{vec}	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
2	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
5	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
10	$-2{,}6270 \pm 0{,}0091$	$-3{,}9170 \pm 0{,}0286$	0,0217
15	$-2{,}6430 \pm 0{,}0138$	$-3{,}9047 \pm 0{,}0286$	0,0219
20	$-2{,}6421 \pm 0{,}0150$	$-3{,}9053 \pm 0{,}0290$	0,0219
30	$-2{,}6420 \pm 0{,}0151$	$-3{,}9053 \pm 0{,}0290$	0,0219

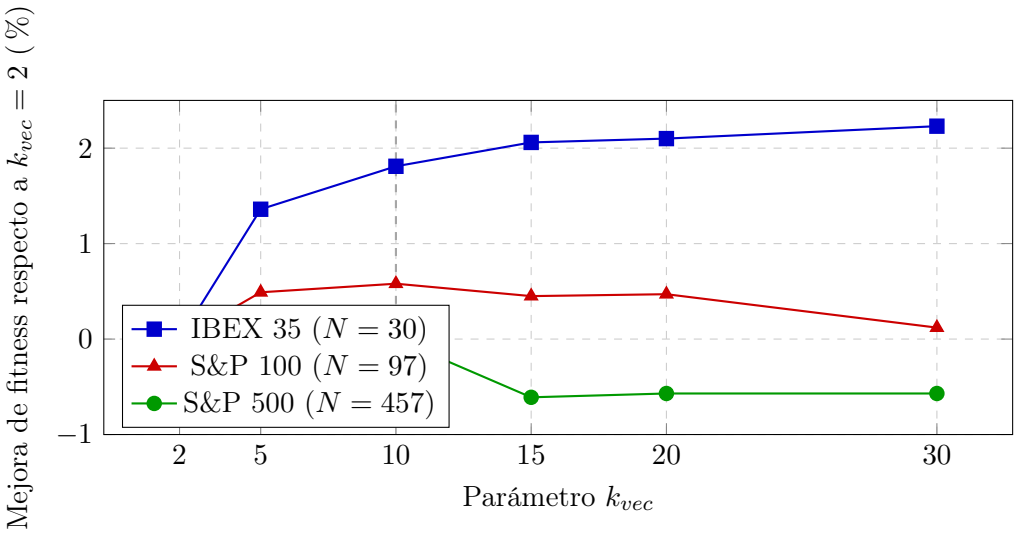


Figura 6: Mejora porcentual del fitness de entrenamiento de SA respecto a $k_{vec} = 2$, en función de k_{vec}. En S&P 500 los valores negativos indican degradación (escala de los ejes): para $k_{vec} \geq 15$ el número de enfriamientos M colapsa a 1, limitando la exploración. En IBEX 35 la mejora es monótona. En S&P 100 hay un máximo en $k_{vec} = 10$.

7.4. Análisis del estudio de k_{vec}

El comportamiento varía notablemente según el dataset:

IBEX 35 ($N = 30$): La mejora es **monotónamente creciente** con k_{vec}. Con max_vec= $k_{vec} \cdot N$, más vecinos por temperatura implica mayor exploración por paso de enfriamiento. Para $N = 30$ el coste es bajo y se puede permitir k_{vec} grande sin que M se vea muy reducido: para $k_{vec} = 30$, $M = \lfloor 10000/900 \rfloor = 11$ (aún suficientes enfriamientos).

S&P 100 ($N = 97$): Máximo en $k_{vec} = 10$ con fitness −2,870. Para $k_{vec} = 30$, max_vec= 2910 y $M = \lfloor 10000/2910 \rfloor = 3$, demasiado pocos enfriamientos para explorar el espacio térmico completo; la calidad cae.

S&P 500 ($N = 457$): **Fenómeno de colapso para** $k_{vec} \geq 15$: max_vec= $15 \times 457 = 6855$ y $M = \text{máx}(1, \lfloor 10000/6855 \rfloor) = 1$. Con solo un enfriamiento, el algoritmo hace una única pasada de hasta 6855 vecinos a temperatura T_0 (sin enfriar) y termina. Los resultados empeoran respecto a $k_{vec} \leq 10$. Para $k_{vec} \in \{2, 5, 10\}$: max_vec$\in \{914, 2285, 4570\}$ y $M \in \{10, 4, 2\}$, dando todos $M \times$ max_vec ≈ 9140 evaluaciones totales — por eso los tres valores producen resultados **idénticamente iguales** (mismas evaluaciones, mismo recorrido estocástico con la misma semilla).

7.5. Conclusiones del estudio de parámetros

1. φ **es irrelevante**: la invarianza de fase de Cauchy lo confirma tanto teórica como empíricamente. El valor predeterminado $\varphi = 0{,}3$ puede mantenerse sin efectos sobre el rendimiento.
2. $k_{vec} = 10$ **es el valor recomendado**: equilibrio óptimo en syp_100, y evita el colapso a $M = 1$ en syp_500. Para N pequeño (ibex_35), valores mayores mejoran ligeramente pero el efecto es marginal.

3. **El factor de colpaso** $M = 1$ aparece cuando max_vec> maxEvals. Para evitarlo, $k_{vec} <$ maxEvals$/N$: con maxEvals= 10 000 y $N = 457$, la cota es $k_{vec} < 21{,}9$, es decir $k_{vec} \leq 21$. El valor $k_{vec} = 10$ mantiene un margen seguro en todos los datasets.